

LFS Push

在 Git 的世界里，你必须先 **commit**（存档），然后才能 **push**（上传到云端）。

如果你修改了文件，直接运行 `git lfs push...`，Git 并不会把你刚刚修改的东西传上去，因为它只认你**已经 commit 过的历史记录**。

为了确保你以后再也不会遇到“以为保存了结果被 reset 掉”的悲剧，请记住这个**雷打不动的标准三步曲**：

永远正确的提交流程

在你修改完 Unreal 项目，想把最新进度备份到 Git 之前，按顺序执行这三步：

第一步：查看状态（可选，但强烈建议养成习惯）

```
Bash
```

```
git status
```

这会用红色的字告诉你，哪些文件被修改了但还没被 Git 记录。

第二步：把所有改动放进暂存区（准备存档）

```
Bash
```

```
git add .
```

注意 `add` 后面有个空格和一点`.`，意思是“把当前目录所有改动都算上”。

第三步：正式 Commit（真正意义上的存档！）

```
Bash
```

```
git commit -m "这里写你这次改了什么，比如：添加了黑市的NPC逻辑"
```

执行完这一步，你的改动才真正变成了像 `sprint1` 那样的存档点！这时候就算你手滑敲了 `reset`，也能通过 `reflog` 找回来。

第四步：推送到远端服务器（云端备份）

```
Bash
```

```
git push origin main
```

(注：通常只要你配置好了 LFS，普通的 `git push` 就会自动把 LFS 的大文件一起传上去，不需要专门敲 `git lfs push`。除非上传中断了你想强制重传 LFS 文件。)

怎么检查自己到底有没有 Commit 成功？

如果你不确定自己刚才到底存没存上，敲这个命令：

```
Bash
```

```
git status
```

- 如果输出写着：`nothing to commit, working tree clean` 🙌 恭喜你！这说明你所有的改动都已经妥妥地 `commit` 进去了，当前工作区是干净的，非常安全。
- 如果输出里有**红色**或**绿色**的文件列表：🙌 警报！这说明你有还没 `commit` 的东西。这时候千万不要 `reset`，赶紧去走上面的 `git add .` 和 `git commit` 流程！

Push All

Step 1: Find your project folder

1. Open up your normal Windows File Explorer (the regular yellow folder icon).
2. Go find the exact folder where your **VertigoGit** project lives. Open that folder so you can see all your game files inside it.

Step 2: Open the "Command Prompt"

1. Look at the very top of that folder window. There is an address bar that shows where you are (it probably says something like `C:\Users\YourName\Documents\VertigoGit`).
2. Click directly on that text in the address bar so it all highlights in blue.
3. Delete all that text, type the letters `cmd`, and press the **Enter** key on your keyboard.
4. A scary-looking black box with white text will pop up. Don't worry, you can't break anything here if you just type what I give you.

Step 3: Send the "Bricks" (The big files)

1. Click inside that black box.
2. Type this exact command (make sure the spaces are right): `git lfs push origin main --all`
3. Press the **Enter** key.
4. **Now, take your hands off the keyboard and wait.** It is currently uploading all those big files to the internet. Depending on your internet speed, this could take a few minutes. Just let it run until it completely finishes and gives you a new, empty typing line at the bottom.

Step 4: Send the "Box" (The final push)

1. Once Step 3 is completely done and the black box is waiting for you again, type this: `git push origin main`
2. Press the **Enter** key.

And that's it! Once that finishes, you can close the black box. You have successfully mailed the package, and your GUI program shouldn't give you that red error anymore.

2. The Task Manager Trick (The best way to know)

We can look under the hood of your computer to see if it is actively sending data to the internet.

1. Right-click on your Windows taskbar at the bottom of the screen and click **Task Manager**.
2. Click on the **Performance** tab on the left side (it looks like a little heartbeat monitor or a zig-zag line).
3. Click on **Wi-Fi** or **Ethernet** (whichever one you use for the internet).
4. Look at the **"Send"** speed. If that number is jumping up and down and is relatively high (like several Mbps), congratulations! It is actively uploading your game files to GitHub. You just need to leave it alone and go grab a coffee.

3. The "Restart" Button (If it's truly stuck)

If you checked your internet speed and it says `0 Kbps` after a few minutes, it might have frozen. You can safely cancel it and try again.

1. Click inside the black Command Prompt box.
2. Hold down the **Ctrl** key on your keyboard and press the letter **C**. (This is the universal "Stop what you are doing" command for the terminal).
3. It should give you a new, fresh typing line.
4. Type the command again: `git lfs push origin main --all` and hit Enter.



If it becomes a mess — you have 3 ways to go back

Option A (BEST PRACTICE): Reset to Version 1

If you want to fully discard Version 2:

```
git log
```

Find the commit hash of Version 1, then:

```
git reset --hard <commit_hash>
```

Then force push if needed:

```
git push origin main --force
```

 This rewrites history on GitHub (fine for solo projects, risky for teams).

Option B (SAFER): Create a “revert commit”

If you already pushed Version 2 and want to undo it cleanly:

```
git revert <commit_hash>  
git push origin main
```

This *does not delete history*, it just adds a “undo” commit.

Option C (BEST FOR EXPERIMENTS): Use a branch

Instead of risking main:

```
git checkout -b version2-experiment
```

Work freely there.

If it fails → delete branch, main stays clean.

If it works → merge it.